kubecost-resize-approvals

Version 1.2.0 · github.com/ttdeangel/kc-lab · Kubecost 3.x · generated from README.md

Approval-gated container right-sizing for any Kubecost 3.x deployment, as a reusable GitHub Action.

Kubecost finds over-requested containers. This package turns each recommendation into a proposal with cost evidence, asks the **workload owner** to approve or disapprove, and lets only the **actioning team** apply it through **Kubecost Actions**. Every step is recorded on a GitHub issue.

```
 plan ──▶ owner: Approve / Disapprove (+ reason) ──▶ actioning team: Apply / Decline ──▶ Kubecost Action ──▶ watch:
applied / failed
          (email · Slack · Teams · webhook · issue)    (work order to actioning only)     (cluster controller)
(one-shot action deleted)
```

- **Uses the Kubecost API only.** Recommendations come from `requestSizingV2`, cost evidence from the Allocation API, and the change goes through `POST /model/action/config`. No kubeconfig or cluster credentials in CI.
- **Owners only decide.** They get the report and choose Approve (optional note) or Disapprove (reason required). They are never pulled into the action.
- **Two gates on applying:** the actor must be in the actioning list, and the job runs in a GitHub Environment where you can require reviewers.
- **Guardrails:**
  - Bad-data increases are blocked; the lab really saw a 302,117m CPU recommendation.
  - Large production cuts need `--force`.
  - Apply re-checks for drift and spec changes.
  - Approvals expire.
  - One-shot actions are cleaned up.
- **Python standard library only.** CI runs on Linux and macOS with Python 3.11 and 3.13.

Verified end to end against Kubecost **3.3.1**: an API-created action resized a Deployment from 500m/512Mi to 10m/20Mi with status `success`. See docs/kubecost-api-notes.md.

`kcctl`: one CLI for everything

```
git clone https://github.com/ttdeangel/kc-lab && cd kc-lab
./kcctl install           # symlink onto your PATH (/usr/local/bin or ~/.local/bin)
kcctl init                # asks for cluster name, port, owner, emails -> lab/lab.env
kcctl bootstrap           # lab up + repo config + GitHub vars/environment + runner + doctor
```

Area	Commands
Settings	`init [--yes]`, `env`, `install [DIR]`, `completion bash|zsh`

Area	Commands
Lab (kind + Kubecost)	`preflight`, `up`, `pause`, `resume`, `destroy`, `runtime`, `ui`, `workloads`, `lab <any kc-lab.sh step>` (e.g. `lab status`, `lab share`)
GitHub wiring	`config`, `secrets`, `runner`, `doctor`, `setup <any setup.sh step>` (e.g. `setup runner-remove`)
Approvals, local	`plan [--read-only]`, `status`, `show`, `approve`, `disapprove --reason`, `apply`, `decline`, `watch [--wait MIN]`. Add `--as USER` to act as someone, `--github` to use the repo's issues instead of the local file
Approvals, in Actions	`run plan [--read-only]`, `run watch`, `runs`, `issues`
End to end	`e2e [ns/name]` (local full flow against the lab), `teardown` (runner + repo variables + cluster)

`kcctl` reads `lab/lab.env` (or `KCCTL_ENV=<file>`), so nothing is tied to a machine or person. It runs on the bash 3.2 that ships with macOS.

Quick start (an existing Kubecost)

1. **Prerequisites in Kubecost:**
   - Actions enabled: `kubecostProductConfigs.actions.enabled: true`, with `config.global.containerRequestRightSizing.enabled: true`.
   - Cluster controller running (free tier needs RSA signing keys).
   - A runner that can reach the Kubecost frontend (`:9090`).
2. **Add the workflow** `examples/workflows/kubecost-resize.yml` to your repo as `.github/workflows/kubecost-resize.yml`.
3. **Add** `.github/kcresize.json`, starting from `examples/config/kcresize.json`:

```
{
  "kubecost": {"cluster": "prod-us-east-1"},
  "scope": {"namespaceLabel": "kcresize.io/approval=required"},
  "owners": {"payments": {"github": ["@acme/payments-leads"], "email": ["payments@acme.com"]}},
  "actioning": {"github": ["@acme/platform"], "email": ["platform@acme.com"]},
  "notify": {"owner": ["email", "slack"], "actioning": ["email", "slack"]}
}
```

4. **Repository settings:**
   - Variables: `KUBECOST_URL`, `KUBECOST_CLUSTER`, and optionally `KCRESIZE_RUNS_ON` (e.g. `["self-hosted","kubecost"]`) and `KCRESIZE_INTERACTIVE=1`.
   - Environment: `kubecost-actioning`, with required reviewers if you want a second sign-off.
   - Secrets: only for the channels you use (see Notifications).
5. **Preview first:** run the workflow with `read-only` to see the plan without writing anything.

Using the Action directly

```
- uses: ttdeangel/kc-lab@v1
  with:
    command: plan                # doctor | plan | event | approve | disapprove | apply | decline | watch | status
    kubecost-url: ${{ vars.KUBECOST_URL }}
    cluster-id: ${{ vars.KUBECOST_CLUSTER }}
    config: .github/kcresize.json
    # role: owner|actioning   (command: event)   read-only: "true"   state: github|file   fail-on-denied: "true"
```

Output	
`result`	`ok` or `denied`
`status`	Proposal status after the command
`plan`, `proposals`, `savings`	Plan id, issues filed, est. monthly savings (`command: plan`)
`action-id`	Kubecost action id (`command: apply`)

The plan and watch steps also write a summary table to the job summary.

How decisions are made

Who	Where	Command
Owner	Issue comment	`/approve [--force] <optional note>` · `/disapprove <reason>`
Owner	Slack / Teams (gateway)	Reason box plus **Approve** / **Disapprove**
Actioning team	Issue comment	`/apply` · `/decline <reason>`
Actioning team	Slack / Teams work order	**Apply via Kubecost** / **Decline**
Anyone allowed	`workflow_dispatch` forms	Owner decision / actioning workflows

- Owners are matched against `owners.<namespace>.github` (users or `@org/team`) and `.ids` (Slack member IDs, emails).
- The actioning team is matched against `actioning.*`. If it's empty, nobody can apply.
- Team (`@org/team`) checks need a `github-token` with `read:org`; the default `GITHUB_TOKEN` can't read team membership.

State machine: `pending → approved | rejected`; `approved → scheduled | declined | expired | stale`; `scheduled → applied | failed`. Details are in docs/architecture.md.

Notifications

Channel	Owner gets	Actioning team gets	Secrets
GitHub issue	Assigned issue with the report and decision commands	Labels and the history table	— (`GITHUB_TOKEN`)
Email	Approval request (HTML)	Work order, FYI, scheduled, applied, failed, expired	`KCRESIZE_SMTP_HOST/PORT/USER/PASSWORD/FROM`

Channel	Owner gets	Actioning team gets	Secrets
Slack	Card (link, or buttons with the gateway)	Work order (Apply / Decline) and status	`KCRESIZE_SLACK_OWNER_WEBHOOK`, `KCRESIZE_SLACK_ACTION_WEBHOOK`
Teams	Adaptive Card (link, or reason box plus buttons via a Workflow)	Work order and status	`KCRESIZE_TEAMS_OWNER_WEBHOOK`, `KCRESIZE_TEAMS_ACTION_WEBHOOK` (Workflows URLs)
Webhook	`approval_requested`	`work_order`, `disapproved`, `scheduled`, `applied`, …	`KCRESIZE_WEBHOOK_URL`, `KCRESIZE_WEBHOOK_SECRET` (HMAC)

Real payloads for every channel are in `examples/notifications/`. Setup for each channel (Gmail app password, Slack app manifest, Teams Workflows, ServiceNow/Jira) is in docs/notifications.md.

Interactive Slack and Teams buttons go through `gateway/`. It's a small stdlib-only service that verifies the Slack signature, an HMAC signature or a token, then forwards the click as a GitHub `repository_dispatch`. It never decides who may do what; the workflows do. A Dockerfile and a k8s manifest are included.

Configuration reference (`.github/kcresize.json`)

Key	Default	Meaning
`kubecost.cluster`	— (or `KUBECOST_CLUSTER`)	Kubecost cluster id
`kubecost.window`	`48h`	Recommendation window; also the action's `rightSizingWindow`
`kubecost.allocationWindow`	`7d`	Cost evidence window
`scope.namespaces` / `scope.namespaceLabel`	all / —	Which namespaces are in scope (`key=value`, read from Kubecost's namespace labels)
`scope.exclude`	system namespaces	Never planned
`profiles.envLabel` / `prodValues` / `prod` / `default`	`env` / `prod,production` / `Production` / `Development`	Which Kubecost profile each namespace uses
`guardrails.*`	10 / 4 / 80 / 20 / 120	`minChangePct`, `maxUpFactor`, `maxDownPctProd`, `driftPct`, `approvalTtlHours`
`owners.<ns>`	—	`github`, `email`, `ids`, `slack` / `teams` (name of a secret holding a per-team webhook)
`actioning`	—	`github`, `email`, `ids`
`notify.owner` / `notify.actioning`	`["email"]`	Channels per audience
`state`	`github`	`github` (issues) or `file` (`stateFile`)

Environment variable	Meaning
`KUBECOST_URL`	Kubecost base URL

Environment variable	Meaning
`KUBECOST_BASIC_USER` / `_PASSWORD` or `KUBECOST_AUTH_HEADER`	Kubecost behind an auth proxy
`KCRESIZE_READ_ONLY`	GET requests only (forced for `demo.kubecost.io`)
`KCRESIZE_ACTION_TIMEOUT_H`	Hours before a scheduled action counts as failed (default 6)
`KCRESIZE_NOTIFY_DRY_RUN`	Write payloads to `.kcresize/outbox/` instead of sending

Command line (local or any CI)

```
export PYTHONPATH=. KUBECOST_URL=http://localhost:9091 KUBECOST_CLUSTER=kcresize-lab
KCRESIZE_CONFIG=.github/kcresize.json
python3 -m kcresize doctor
KCRESIZE_STATE=file python3 -m kcresize plan
python3 -m kcresize approve tbm-payments/fraud-scorer --reason "traffic is flat" --actor fraud-lead
python3 -m kcresize apply tbm-payments/fraud-scorer --actor platform-oncall
python3 -m kcresize watch
```

Exit codes: `0` ok, `1` error, `3` denied.

For read-only data testing against the public demo:

```
KUBECOST_URL=https://demo.kubecost.io KUBECOST_CLUSTER=kc-demo-prod KCRESIZE_NAMESPACES=kubecost-v3
KCRESIZE_STATE=file python3 -m kcresize plan
```

Lab: spin up, test, spin down

`lab/` builds a kind + Kubecost 3.3.1 lab on macOS (Colima) or EC2, driven by `kcctl`.

- **Settings:** every name (cluster, port, people, runner) comes from `lab/lab.env`.
- **Side by side:** the defaults (cluster `kcresize-lab`, port 9091, work dir `.lab/<cluster>`) run it next to an existing lab without touching it.

Spin up (about 15 min, then a few hours of usage data before recommendations appear)

```
brew install colima docker kind kubectl helm gh jq && gh auth login
git clone https://github.com/ttdeangel/kc-lab && cd kc-lab
./kcctl install && kcctl init      # cluster, port, owner + actioning user/email
kcctl preflight                    # VM CPU / memory / disk, port free, other clusters
kcctl bootstrap                    # up + config + GitHub vars/environment + runner + doctor
git add .github/kcresize.json && git commit -m "lab config" && git push
```

For a faster first run, set `"window": "6h"` under `kubecost` in `.github/kcresize.json`.

End-to-end test

#	Do	Expect
1	`kcctl run plan --read-only`, then `kcctl run plan`	Summary table, then one issue per workload plus approval emails
2	On `tbm-payments/fraud-scorer`: `/disapprove`	"A reason is required"
3	`/approve traffic is flat`	`kcresize:approved`, work order to the actioning team
4	`/apply` (from the actioning team)	`kcresize:scheduled`, the action shows in Kubecost under Savings → Actions
5	Wait about 70 min (`kcresize - watch` runs every 15 min)	`kcresize:applied`, issue closed, the action deleted
6	`kubectl --context kind-<cluster> -n tbm-payments get deploy fraud-scorer -o jsonpath='{..requests}'`	`{"cpu":"10m","memory":"20Mi"}`
7	On `shop-prod/api`: `/approve`, then `/approve --force`	The first is refused (large production cut), the second goes through

To check without GitHub, `kcctl e2e` runs plan → approve → apply → wait, using local state.

Spin down and up

Level	Down	Up	Keeps
Pause	`kcctl pause`	`kcctl resume`	Everything, including Kubecost history
Kubecost only	`kcctl lab uninstall`	`kcctl lab objectstore secrets kubecost expose`	Cluster and workloads
Full teardown	`kcctl teardown`, then `colima stop`	`kcctl up` → `kcctl setup github` → `kcctl runner` → `kcctl doctor`	`lab/lab.env`, the signing keys, the repo and issues

Pause refuses while a resize is in flight (`--force` overrides). It also:

1. Removes the repo variables so the scheduled workflows skip.
2. Stops the cluster.
3. Stops Colima if no other kind cluster is running.

`kcctl resume` reverses that and runs `doctor`. For a full teardown, the order is: no resize in flight → `setup unconfigure` → `setup runner-remove` → `destroy` → `colima stop`.

Repository layout

```
kcctl                       CLI for the lab, GitHub wiring and approvals
action.yml                  reusable composite Action (entry: python -m kcresize)
kcresize/                   package: kubecost.py (API client) planner.py (guardrails)
                            workflow.py (state machine) state.py (GitHub issues / file)
                            report.py (renderers) notify.py (email/Slack/Teams/webhook) cli.py
.github/workflows/          plan · owner decision · actioning (environment-gated) · watch · ci
.github/kcresize.json       this repo's lab config
examples/                   consumer workflow, config, sample payloads for every channel, Slack app manifest
gateway/                    Slack/Teams/webhook → repository_dispatch relay (+ Dockerfile, k8s.yaml)
lab/                        kind + Kubecost lab, sample workloads, setup, local e2e
docs/                       architecture, notifications, verified Kubecost API notes
tests/                      unit + end-to-end tests against in-process Kubecost and GitHub stand-ins
```

Development

```
python3 -m unittest discover -s tests -v     # no network, no dependencies
```

CI runs the tests on Ubuntu and macOS (Python 3.11 and 3.13). It also smoke-tests `action.yml` itself against the mock Kubecost: plan → approve → apply → watch.

Limits and caveats

- **Timing:** Kubecost runs new actions on its own schedule. In the lab it took about 70 minutes from apply to resize. `watch` runs every 15 minutes.
- **Lagging requests:** `latestKnownRequest` lags after a resize. kcresize verifies through `/model/actions/completed` instead.
- **Realized savings:** `/model/actions/savings` needs cost history from before the resize.
- **Scope:** Deployments only for now (`scope.controllerKinds`).
- **Free tier:** Kubecost limits Actions to 250 cores.
- **Overlapping actions:** another enabled Kubecost request-sizing action that targets the same namespaces bypasses approvals. `doctor` warns about it.

License

MIT. See LICENSE.